

PYTHON TRADING STRATEGY BACKTESTER

Detailed Strategy Specification, Project Architecture, Scrum Specifications and Phase-wise Sprint Plan

Document	Project Blueprint v1.0
Strategy scope	EURUSD, GBPUSD, AUDUSD, XAUUSD, XAGUSD
Primary timeframes	15-minute structure / 1-minute execution
Sprint start	31 August 2026
Sprint cadence	10 workdays per sprint
Capacity model	1 story = 2 person-hours; planned work rate = ~1 hour/day
Initial cost target	₹0 / free-first development stack

Purpose. This document converts the discretionary trading method into an engineering specification suitable for a sequential, no-look-ahead Python backtester. It also defines the software architecture and a practical Scrum delivery plan.

Prepared: 29 August 2026

Table of Contents

1. Executive Summary
2. Strategy Scope and Trading Universe
3. Trading Session and Time Handling
4. Swing High / Swing Low Specification
5. 15-Minute MSS and Swing Invalidation
6. OTE Retracement Logic
7. 1-Minute Confirmation Logic
8. BISI / SIBI Detection and Entry Execution
9. Stop-Loss and Risk Filters
10. Take-Profit Hierarchy
11. Complete Long and Short Decision Flows
12. Assumptions Still Requiring Formalization
13. Backtester Design Principles
14. Project Architecture and Module Responsibilities
15. State-Machine Design
16. Event Timeline and Look-Ahead Protection
17. Trade Simulation and Intrabar Ambiguity
18. Data Model and Result Schema
19. Testing and Validation Strategy
20. Cost and Technology Stack
21. Scrum Framework and Capacity
22. Sprint 1 Detailed Plan
23. Phase-wise Multi-Sprint Roadmap
24. Definitions of Done and Acceptance Gates
25. Risks, Controls and Future Enhancements

1. Executive Summary

The objective is to build a Python backtesting system that reproduces the user's manual strategy as faithfully as possible while preventing future-data leakage. The strategy begins with 15-minute market structure, waits for a confirmed market structure shift (MSS), requires a retracement of at least 60% of the relevant swing range, then moves to 1-minute structure for a second MSS and BISI/SIBI-based entry. Entry occurs at the opening of the next 1-minute candle after the retest candle confirms that the imbalance remains respected.

The architecture is intentionally sequential and stateful. Pandas is used for loading, cleaning and resampling data; the strategy itself is executed candle by candle through a state machine. This allows the backtest to behave like a trader receiving information over time rather than a script scanning the completed chart after the fact.

Delivery expectation: a basic working backtester is expected after roughly four implementation sprints, while advanced target logic, analytics and deeper validation extend the roadmap to approximately six sprints. At one hour of work per workday, a trustworthy version is expected to require roughly 50-66 focused hours.

2. Strategy Scope and Trading Universe

The initial implementation supports five instruments:

- EURUSD
- GBPUSD
- AUDUSD
- XAUUSD
- XAGUSD

The system is designed around two timeframes:

Timeframe	Role
15-minute	Primary market structure, swing identification, MSS, OTE anchor, structural stop reference and higher-timeframe swing confirmation
1-minute	Lower-timeframe swing/MSS confirmation, BISI/SIBI detection, ATR buffer, entry confirmation and execution

The first version should run one independent strategy engine per symbol. The recommended V1 constraint is one active setup and one open trade per symbol at a time. This avoids state collisions while keeping the architecture extensible.

3. Trading Session and Time Handling

The intended trading window is 09:30-16:00 in the UTC-4 market clock used by the strategy. The user also expressed the equivalent working window as approximately 19:00-01:30 IST. The implementation should not permanently hardcode this equivalence because daylight-saving and source timestamp conventions can create offset errors.

Recommended engineering rule: store timestamps internally in UTC, attach explicit timezone information to source data, and convert into the strategy's session timezone only when evaluating session eligibility.

Pre-session swings remain valid. Therefore the engine must process sufficient historical candles before the session opens so that a valid confirmed 15-minute swing may already exist at 09:30.

4. Swing High / Swing Low Specification

A candle is a swing only when it dominates the three candles immediately before it and the three candles immediately after it.

Swing High

For candle i:

```
high[i] > high[i-1], high[i-2], high[i-3]
AND
high[i] > high[i+1], high[i+2], high[i+3]
```

Swing Low

```
low[i] < low[i-1], low[i-2], low[i-3]
AND
low[i] < low[i+1], low[i+2], low[i+3]
```

A crucial backtesting distinction is the difference between the swing candle time and the confirmation time. Because three subsequent candles are required, the engine is not allowed to know that candle i is a swing until candle i+3 has closed.

Field	Meaning
swing_time	Timestamp of the actual pivot candle.
confirmation_time	Timestamp when the third following candle closes and the swing becomes knowable.
price	High for a swing high; low for a swing low.
status	Active, invalidated, touched, untouched, broken, etc.

5. 15-Minute MSS and Swing Invalidation

A bullish MSS is confirmed only when a 15-minute candle closes above the active swing high. A bearish MSS is confirmed only when a 15-minute candle closes below the active swing low. A wick alone is insufficient.

Direction	MSS condition
Bullish	15m close > active swing-high price
Bearish	15m close < active swing-low price

Wick-break invalidation rule

If price wicks through a swing but the candle fails to close through it, the engine gives the immediately following 15-minute candle one opportunity to confirm the break. If that next candle also fails to close beyond the swing, that swing is invalidated and the strategy waits for another valid confirmed swing.

Bullish-side example:

```
wick above swing high -> no close above -> inspect immediately next 15m candle -> if next close > swing high: MSS; otherwise invalidate that swing high
```

The bearish case is the exact inverse.

6. OTE Retracement Logic

After the 15-minute MSS, the system calculates the retracement using the relevant 15-minute swing range. The user confirmed that the setup becomes eligible once price retraces at least 60% of the range.

Bullish example

```
Swing Low = 1.1000
Broken Swing High = 1.1100
Range = 0.0100 = 100 pips
60% retracement = 60 pips
OTE threshold = 1.1100 - 0.0060 = 1.1040
```

Bullish formula:

```
ote_long = swing_high - ((swing_high - swing_low) * 0.60)
OTE reached when price <= ote_long
```

Bearish formula:

```
ote_short = swing_low + ((swing_high - swing_low) * 0.60)
OTE reached when price >= ote_short
```

This specification treats OTE as a threshold rather than a narrow Fibonacci band: once the retracement reaches 60% or deeper, the 1-minute confirmation process may begin.

7. 1-Minute Confirmation Logic

After OTE is reached, the engine activates 1-minute market-structure analysis. The same three-before / three-after swing definition is used on 1-minute candles, including the same confirmation delay principle.

Higher-timeframe bias	1-minute confirmation
Bullish	Identify relevant 1m swing high and require a 1m candle close above it.
Bearish	Identify relevant 1m swing low and require a 1m candle close below it.

Only after this lower-timeframe MSS does the strategy begin waiting for the appropriate imbalance: BISI for longs and SIBI for shorts.

8. BISI / SIBI Detection and Entry Execution

BISI and SIBI are defined using a three-candle imbalance relationship.

BISI - bullish imbalance

```
high[candle 1] < low[candle 3]
Zone bottom = high[candle 1]
Zone top = low[candle 3]
```

SIBI - bearish imbalance

```
low[candle 1] > high[candle 3]
Zone bottom = high[candle 3]
Zone top = low[candle 1]
```

The strategy does not enter when the imbalance first forms. It waits for price to retrace back into the imbalance.

Long confirmation

- 1 Bullish 1-minute MSS has already occurred.

- 2 A BISI forms.
- 3 A later 1-minute candle retraces into the BISI.
- 4 The interacting candle may wick below the BISI, but it must not close below the BISI lower boundary.
- 5 If the candle respects that rule, the long entry is executed at the open of the next 1-minute candle.

Short confirmation

- 1 Bearish 1-minute MSS has already occurred.
- 2 A SIBI forms.
- 3 A later 1-minute candle retraces into the SIBI.
- 4 The interacting candle may wick above the SIBI, but it must not close above the SIBI upper boundary.
- 5 If respected, the short entry is executed at the open of the next 1-minute candle.

The next-candle-open rule is essential for realistic simulation: the strategy cannot know whether the retest candle closed validly until that candle is complete.

9. Stop-Loss and Risk Filters

The stop is anchored to the relevant 15-minute structural swing and extended by two times the 1-minute ATR.

Direction	Stop formula
Long	SL = relevant 15m swing low - (2 x 1m ATR)
Short	SL = relevant 15m swing high + (2 x 1m ATR)

Maximum permitted stop distance:

Instrument class	Limit
EURUSD / GBPUSD / AUDUSD	10 pips maximum
XAUUSD / XAGUSD	25 ticks maximum

If the structural stop plus ATR buffer exceeds the permitted maximum distance from entry, the setup is rejected. Instrument metadata must explicitly define pip size and tick size so that these limits are converted correctly.

The ATR period has not yet been explicitly chosen by the user. A common default such as ATR(14) may be used only after it is deliberately accepted as a strategy parameter; it should not be silently embedded.

10. Take-Profit Hierarchy

The minimum acceptable target is 2R, where R is the absolute distance between entry and stop loss.

Long: $TP_{2R} = entry + 2 \times (entry - stop)$
 Short: $TP_{2R} = entry - 2 \times (stop - entry)$

The strategy may instead target the second untouched 15-minute swing in the trade direction when that provides a larger reward. For longs, this means the second untouched swing high above entry; for shorts, the second untouched swing low below entry.

Round-number override

Round levels occur every 100 pips. If 2R is achievable and the second untouched swing is farther away, but a qualifying round-number support/resistance level exists between 2R and that second swing, the strategy takes the conservative 2R target rather than holding for the distant structural target.

The phrase 'has acted as support/resistance in the past' still requires a mathematical definition. Until that rule is formalized, the advanced round-number target selector should remain a separate module and should not be allowed to contaminate the validity of the core entry backtest.

11. Complete Long and Short Decision Flows

Long flow

```
Maintain confirmed 15m swings
-> wait for 15m close above active swing high
-> bullish 15m MSS
-> calculate 60% retracement threshold
-> wait for price to reach threshold or deeper
-> activate 1m swing tracking
-> wait for 1m close above relevant swing high
-> bullish 1m MSS
-> wait for BISI
-> wait for later retracement into BISI
-> retest candle must not close below BISI bottom
-> queue long entry
-> enter at next 1m open
-> calculate 15m swing-low minus 2x 1m ATR stop
-> reject if stop > instrument limit
-> calculate 2R and advanced target candidate
-> simulate until SL/TP/other exit rule
```

Short flow

```
Maintain confirmed 15m swings
-> wait for 15m close below active swing low
-> bearish 15m MSS
-> calculate 60% retracement threshold
-> wait for price to reach threshold or deeper
-> activate 1m swing tracking
-> wait for 1m close below relevant swing low
-> bearish 1m MSS
-> wait for SIBI
-> wait for later retracement into SIBI
-> retest candle must not close above SIBI top
-> queue short entry
-> enter at next 1m open
-> calculate 15m swing-high plus 2x 1m ATR stop
-> reject if stop > instrument limit
-> calculate 2R and advanced target candidate
-> simulate trade
```

12. Assumptions Still Requiring Formalization

The strategy is sufficiently defined to begin building the core engine, but the following items should remain explicit backlog decisions rather than hidden assumptions:

Item	Current status / required decision
ATR period	Multiplier is fixed at 2x; ATR lookback period still needs explicit confirmation.
Relevant 15m swing pair for OTE	Current interpretation: the structural low/high associated with the swing broken by MSS. Edge cases need
Untouched swing definition	Recommended: no subsequent candle has traded at or through the swing price after confirmation; user

Item	Current status / required decision
Historical round-level significance	Need objective lookback, tolerance, minimum touches/rejections and reaction size.
Session-end behavior	Need decision on whether pending setups are cancelled at 16:00 and whether existing trades may remain open.
Same-candle SL and TP	OHLC cannot reveal intrabar order; conservative policy recommended unless lower-resolution/tick data is available.
Spread/slippage/commission	V1 may omit, but realistic performance should later include them.

13. Backtester Design Principles

- Sequential simulation: time moves forward one candle at a time.
- No look-ahead: a swing is usable only after its confirmation time.
- Separation of concerns: data preparation, indicators, strategy state, execution and analytics are independent layers.
- Deterministic rules: every discretionary phrase must eventually become a measurable condition.
- Traceability: every setup should record why it advanced, failed, was invalidated or was rejected.
- Testability: each module must be testable independently before end-to-end integration.
- Free-first: local Python and open-source libraries are sufficient for V1.

The central design idea is that the backtester should behave like a trader with memory. At each candle it asks: What information is currently knowable? What setup state am I in? What condition am I waiting for? Did this candle satisfy or invalidate it?

14. Project Architecture and Module Responsibilities

Recommended repository structure:

```
trading_backtester/ |-- config/ | |-- settings.py | `-- instruments.py |-- data/ | |-- raw/ | `--
processed/ |-- core/ | |-- candle.py | |-- events.py | |-- state.py | `-- models.py |-- data_engine/ |
|-- loader.py | |-- cleaner.py | |-- timezone.py | `-- resampler.py |-- indicators/ | |-- swings.py |
|-- atr.py | `-- imbalance.py |-- strategy/ | |-- mss.py | |-- ote.py | |-- confirmation.py | |--
round_levels.py | `-- strategy_engine.py |-- execution/ | |-- entry.py | |-- stop_loss.py | `--
take_profit.py |-- backtest/ | |-- engine.py | |-- trade_manager.py | |-- portfolio.py | `-- metrics.py
|-- results/ |-- tests/ |-- main.py `-- requirements.txt
```

Module responsibilities

Module	Responsibility
config/settings.py	Session, swing window, OTE threshold, ATR multiplier, minimum RR and other global parameters.
config/instruments.py	Per-symbol pip/tick metadata and maximum stop limits.
data_engine/loader.py	Read CSV/Parquet/database data into a standard schema.
data_engine/cleaner.py	Sort, deduplicate, validate OHLC and report gaps.
data_engine/timezone.py	Normalize timestamps and evaluate trading-session time.
data_engine/resampler.py	Build aligned 15m candles from 1m source data.
indicators/swings.py	Detect and confirm 1m/15m swings without look-ahead.
indicators/atr.py	Calculate 1m ATR used by stop logic.

Module	Responsibility
indicators/imbalance.py	Detect BISI/SIBI zones and lifecycle.
strategy/mss.py	Track active swings, wick breaks, close-based MSS and invalidation.
strategy/ote.py	Calculate and monitor the 60% retracement threshold.
strategy/strategy_engine.py	Own the setup state machine and coordinate strategy events.
execution/*	Translate a valid signal into entry, SL and TP instructions.
backtest/engine.py	Advance the simulation candle by candle.
backtest/trade_manager.py	Open, monitor and close simulated trades.
backtest/metrics.py	Compute performance statistics.

15. State-Machine Design

The strategy should not be implemented as one large nested conditional. Each symbol maintains an explicit state. A suggested lifecycle is:

```
IDLE / WAITING_FOR_15M_MSS
-> WAITING_FOR_OTE
-> WAITING_FOR_1M_MSS
-> WAITING_FOR_IMBALANCE
-> WAITING_FOR_RETEST
-> ENTRY_QUEUED
-> IN_TRADE
-> TRADE_CLOSED / RESET
```

Additional temporary state may be required for a wick break:

```
WAITING_FOR_15M_MSS -> WICK_BREAK_PENDING -> MSS_CONFIRMED or SWING_INVALIDATED
```

Each transition should log the symbol, timestamp, old state, new state, triggering event and relevant price levels. This makes manual chart reconciliation dramatically easier.

16. Event Timeline and Look-Ahead Protection

The engine should use 1-minute data as the source of truth and derive 15-minute bars from it. At every 1-minute step, it updates the current 1-minute logic. When a 15-minute boundary closes, it finalizes that 15-minute candle and updates higher-timeframe logic.

Illustrative sequence:

Time	Event / state consequence
10:30	15m candle wicks above swing high but closes below -> WICK_BREAK_PENDING.
10:45	Next 15m candle also fails to close above -> invalidate swing.
11:00	A newly confirmable swing becomes available.
11:30	15m close above active swing -> bullish MSS.
11:47	1m price reaches $\geq 60\%$ retracement -> activate 1m confirmation.
11:55	1m close above relevant 1m swing -> bullish 1m MSS.
11:58	BISI forms.
12:03	Retest candle touches BISI and closes above its lower boundary -> entry queued.

Time	Event / state consequence
12:04	Enter long at next 1m open.

A precomputed DataFrame may contain future columns for convenience, but the strategy engine must never access a signal before its legal confirmation timestamp. Tests should explicitly assert this.

17. Trade Simulation and Intrabar Ambiguity

OHLC candles do not reveal the exact path between open, high, low and close. If both stop and target lie within the same 1-minute candle, the backtester cannot know which was hit first from OHLC alone.

Recommended V1 policy: conservative execution. If both SL and TP are reachable in the same candle and no lower-resolution data resolves the order, assume the adverse outcome (SL first). Make this configurable so a later tick-data engine can replace it.

- Entry signals are confirmed at candle close and executed at the next candle open.
- Open trades are checked on each new candle for SL/TP interaction.
- Spread, slippage and commission should be configurable future extensions.
- Trade execution assumptions must be written into the backtest report so results are reproducible.

18. Data Model and Result Schema

Suggested core objects:

```
Swing(type, price, swing_time, confirmation_time, status) MSS(direction, broken_swing_id, time,
close_price) Imbalance(type, bottom, top, formed_at, status) Setup(symbol, direction, state, mss_15m,
ote_level, mss_1m, imbalance) Trade(symbol, direction, entry_time, entry_price, stop_loss,
take_profit, exit_time, exit_price, result, r_multiple)
```

Recommended trade log columns:

Category	Fields
Identity	trade_id, symbol, direction
Entry	signal_time, entry_time, entry_price
Structure	15m_mss_time, 15m_broken_swing, ote_level, 1m_mss_time
Imbalance	bisi_sibi_time, zone_bottom, zone_top, retest_time
Risk	atr_1m, structural_stop, final_stop, stop_distance_pips_or_ticks
Target	tp_2r, second_untouched_swing, chosen_tp, target_reason
Exit	exit_time, exit_price, result, r_multiple
Diagnostics	setup_id, invalidation_reason, execution_policy

19. Testing and Validation Strategy

Testing must happen layer by layer. A feature is not considered done merely because it runs.

Layer	Validation
Data	Check ordering, duplicates, missing candles, timezone and OHLC integrity.
Resampling	Manually calculate sample 15m OHLC from constituent 1m candles.

Layer	Validation
Swings	Compare detected pivots with chart examples and verify confirmation delay.
MSS	Verify close-vs-wick cases, including immediate-next-candle invalidation.
OTE	Use hand-calculated ranges such as 1.1000-1.1100 -> 1.1040 threshold.
1m MSS	Verify no 1m confirmation is used before OTE activation.
BISI/SIBI	Test formation, retest, invalid close and valid next-open entry.
Risk	Check pip/tick conversion and max-SL rejection.
Execution	Test TP-only, SL-only and same-candle TP+SL scenarios.
End-to-end	Replay selected historical days and compare every state transition to manual chart review.

Automated tests should use pytest. Manual visual validation should remain part of the process because the goal is to reproduce a discretionary strategy, not merely satisfy code assertions.

20. Cost and Technology Stack

The project can be developed locally at essentially zero software cost.

Component	Recommended option	Cost
Language	Python	Free
Data manipulation	pandas, NumPy	Free
Testing	pytest	Free
Storage V1	CSV / Parquet / SQLite	Free
Storage later	Local PostgreSQL	Free
Charts	Matplotlib	Free
IDE	VS Code	Free
Version control	Git + GitHub	Free tiers available
Analytics later	Power BI Desktop / PostgreSQL queries	Free/local options

The major potential future cost is high-quality historical tick or 1-minute data. The recommendation is not to buy data until the strategy engine works correctly on the historical data already available.

21. Scrum Framework and Capacity

Sprint model:

Parameter	Value
Sprint start	31 August 2026
Sprint length	10 workdays
Developer capacity	~1 hour/day
Nominal sprint capacity	10 hours
Story sizing convention	1 story = 2 person-hours
Theoretical maximum	5 stories

Parameter	Value
Recommended commitment	4 stories = 8 hours
Reserved buffer	2 hours for debugging, learning, testing and spillover

Scrum principle: the sprint goal is more important than filling every hour. If a foundational story takes longer, scope is protected by dropping optional work rather than declaring partially tested work complete.

Definition of a story

A story is a vertical, demonstrable increment with acceptance criteria. Coding tasks, tests and documentation belong inside the story rather than being treated as optional work after implementation.

22. Sprint 1 Detailed Plan - Foundation & Market Structure

Sprint Goal: Turn raw 1-minute historical data into reliable 15-minute candles and correctly identify confirmed 15-minute swings without look-ahead bias. If capacity remains, prototype MSS detection.

Committed Story 1 - Project Foundation (2h)

User story: As a developer, I want a clean project structure and configuration system so that the backtester can be built and tested incrementally.

- Create repository/folder structure.
- Create and verify virtual environment.
- Create requirements.txt.
- Create configuration modules.
- Add logging.
- Create main.py smoke run.
- Add initial README and Git baseline.

Done when: project starts successfully, imports work, configuration loads, logging works and the repository has a clean initial commit.

Committed Story 2 - Historical Data Pipeline (2h)

User story: As the backtester, I want clean chronological 1-minute OHLC data so that every later strategy calculation operates on reliable input.

- Inspect current datasets and source timezone.
- Standardize columns.
- Parse timestamps.
- Sort chronologically.
- Remove/report duplicates.
- Validate OHLC relationships.
- Report gaps/missing candles.
- Produce data-quality summary.

Done when: data loads into the standard schema and the quality report exposes date range, row count, duplicates, gaps and invalid OHLC rows.

Committed Story 3 - 15-Minute Resampling (2h)

- Define exact 15m boundaries.
- Open = first 1m open.
- High = max constituent high.
- Low = min constituent low.
- Close = last 1m close.
- Manually verify selected 15m candles.

Done when: generated 15m candles are correctly aligned and sample periods match manual calculations.

Committed Story 4 - Swing Detection Engine (2h)

- Create Swing model.
- Implement 3-before/3-after high detector.
- Implement inverse low detector.
- Track swing time and confirmation time.
- Write unit tests.
- Print and manually verify sample swings.
- Ensure no strategy component can consume a swing before confirmation.

Done when: swing highs/lows match manual examples and look-ahead tests pass.

Optional / Buffer Story - MSS Prototype (up to 2h)

Only pull this into the sprint if all four committed stories are genuinely done. Prototype active-swing tracking and close-based MSS; full wick-invalidation behavior may remain for Sprint 2.

Sprint 1 Day-by-Day Plan

Workday	Primary focus	Expected checkpoint
Day 1 - Aug 31	Story 1: foundation	Venv, folders, config, logging and main.py run.
Day 2	Finish Story 1 / inspect data	Repository baseline complete; source columns/timezone understood.
Day 3	Story 2: loader + cleaner	Clean 1m DataFrame produced.
Day 4	Story 2: validation	Data-quality report and edge cases tested.
Day 5	Story 3: resampling	1m -> 15m generation working.
Midpoint review	Inspect commitment health	Stories 1-3 status checked; do not add scope if behind.
Day 6	Story 3: manual validation	Sample 15m bars verified against source 1m data.
Day 7	Story 4: swing detector	Mathematical swing logic implemented.
Day 8	Story 4: confirmation timing	Swing time vs confirmation time tracked and tested.
Day 9	Validation / bug fixing	Compare detector against selected chart sections.
Day 10	Sprint review + retrospective	Demo pipeline; document lessons; optionally show MSS prototype.

23. Phase-wise Multi-Sprint Roadmap

Sprint	Goal	Indicative stories / outputs
Sprint 1	Foundation & Swings	Project setup; data pipeline; 15m resampling; swing engine; optional MSS prototype.
Sprint 2	15m Market Structure & OTE	Active swings; close-based MSS; wick invalidation; session filtering; 60% OTE; state transi
Sprint 3	1m Entry Logic	1m swings; 1m MSS after OTE; BISI/SIBI; retest validation; next-open entry signal.
Sprint 4	Risk & Core Trade Simulation	ATR; structural SL; pip/tick filters; 2R TP; trade lifecycle; conservative intrabar policy.
Sprint 5	Advanced Targets & Validation	Second untouched swing; 100-pip round levels; historical S/R significance; broader chart r
Sprint 6	Analytics & Optimization	Trade log; win rate; expectancy; profit factor; drawdown; pair/time analysis; database/rep

Estimated total effort

Stage	Stories	Hours
Sprint 1	4-5	8-10
Sprint 2	5-6	10-12
Sprint 3	5-6	10-12
Sprint 4	4-5	8-10
Sprint 5	4-6	8-12
Sprint 6	3-5	6-10
Total	25-33	50-66

At approximately one focused hour per workday, this corresponds to roughly 50-66 working days. A basic end-to-end backtester should exist after Sprint 4; Sprints 5-6 improve fidelity, target logic and analytical usefulness.

24. Definitions of Done and Acceptance Gates

Global Definition of Done

- Code is implemented and readable.
- Automated tests exist for deterministic logic.
- Representative edge cases pass.
- No known look-ahead violation exists.
- Manual sample validation is completed where chart interpretation matters.
- Configuration values are not silently hardcoded when they should be parameters.
- Logging is sufficient to reconstruct why a setup advanced or failed.
- README / project notes are updated for material behavior changes.
- Changes are committed to version control.

Release gates

Gate	Requirement before proceeding
Data gate	No unexplained timezone/resampling errors.
Swing gate	Manual chart samples agree and confirmation delay is proven.
MSS gate	Wick vs close and invalidation examples behave correctly.
Entry gate	Signal occurs only after OTE -> 1m MSS -> imbalance -> valid retest -> next open.
Risk gate	Pip/tick conversion and max-stop rejection verified for every instrument.
Backtest gate	Trade simulator handles normal and ambiguous candles deterministically.
Trust gate	Selected historical sessions are replayed event-by-event against manual interpretation.

25. Risks, Controls and Future Enhancements

Risk	Control
Look-ahead bias	Store confirmation timestamps; event-driven access; explicit unit tests.
Timezone/session mismatch	UTC normalization and timezone-aware session conversion.
Wrong 15m alignment	Manual constituent-candle checks and source-specific boundary configuration.
Discretion hidden in code	Maintain an assumptions register; do not silently invent rules.
Overfitting	Do not optimize parameters until the base rules are stable; use out-of-sample periods later.
Unrealistic fills	Next-open entries; conservative same-candle SL/TP policy; later add spread/slippage.
Metal tick ambiguity	Define tick size per data source before enforcing 25-tick stop limit.
Premature complexity	Keep advanced round-level logic separate until core engine is validated.

Future enhancements

- Spread, commission and slippage models.
- Tick-data or lower-resolution intrabar replay.
- Portfolio-level risk and correlated-position controls.
- Parameter experiment runner with strict out-of-sample validation.
- PostgreSQL trade warehouse and Power BI analytics.
- Visual trade replay showing swings, MSS, OTE, imbalance, entry, SL and TP.
- Automated discrepancy reports comparing strategy events against manually labelled examples.

Final engineering principle

Build one layer, prove it, then add the next. The fastest path to a trustworthy backtester is not writing the most code; it is preventing an early structural mistake from propagating through every later module.

The immediate next implementation milestone is Sprint 1, Story 1: create the project foundation, then audit the historical 1-minute data before any strategy logic is written.